WEB APPLICATION

PENETRATION TEST REPORT

OWASP Juice Shop

Grey-Box Web Application Security Assessment

FINDINGS AT A GLANCE

1	3	1
CRITICAL	HIGH	MEDIUM

Prepared by	**Selvi Melani**
Role	Web Application Penetration Tester
Target	OWASP Juice Shop (local lab instance)
Assessment date	14 October 2026
Report version	1.0 — Final
Classification	**PORTFOLIO SAMPLE**

NOTE — This document is a portfolio sample produced against an intentionally vulnerable application.

Disclaimer and Authorization

This document contains the results and supporting evidence of a web application penetration test. It is intended solely for the recipient and authorized stakeholders. The information herein describes security weaknesses that could be abused if disclosed, and should be handled accordingly.

Authorized testing. All testing described in this report was performed against OWASP Juice Shop, an intentionally vulnerable application published by the OWASP Foundation for security education. The target was run as a private, self-hosted instance on the tester's own machine. No third-party or production system was accessed at any point.

Purpose. This report is a professional portfolio sample. It demonstrates the tester's methodology, hands-on exploitation skills, evidence collection and reporting style using a legal practice target.

Document Control

Version	Description	Date
0.9	Initial draft — findings and evidence compiled	14 Oct 2026
1.0	Final report	14 Oct 2026

Table of Contents

Disclaimer and Authorization 2

1. Executive Summary 4

2. Findings Overview 5

3. Scope and Methodology 6

4. Technical Findings 7

F-01 SQL Injection — Authentication Bypass 7

F-02 Unauthenticated Admin Panel Access 9

F-03 IDOR on Shopping Basket 10

F-04 Sensitive File Directory Exposed 12

F-05 DOM-Based Cross-Site Scripting 14

5. Remediation Summary 16

Appendix A — Test Environment Setup 17

1. Executive Summary

A grey-box penetration test was performed against the OWASP Juice Shop web application to identify security vulnerabilities in its storefront, REST API and administrative interface. Testing combined automated reconnaissance with manual exploitation using an intercepting proxy.

The application is at HIGH overall risk. Five vulnerabilities were identified across three OWASP Top 10 (2021) categories: one Critical, three High and one Medium. Most significantly, a SQL injection flaw in the login form allows an attacker to authenticate as any user without a password. Chained with weak access control on the administration area, this leads to full disclosure of every customer's email address, a complete compromise of customer data.

1.1 Key Findings

- **Authentication bypass via SQL injection** (Critical): login as administrator with no valid credentials.
- **Unrestricted access to the administration panel** (High): all registered users and feedback exposed.
- **Insecure Direct Object Reference on the basket API** (High): read other users' baskets by changing an ID.
- **Sensitive file directory exposed via robots.txt** (High): backups and a password vault reachable.
- **DOM-based Cross-Site Scripting in search** (Medium): arbitrary JavaScript executes in the browser.

1.2 Risk Distribution

Severity	Count	Indicative remediation window
Critical	1	Within 7 days
High	3	Within 14–30 days
Medium	1	Within 30–90 days
Total	5	

2. Findings Overview

Findings are listed in order of severity. Each is detailed in Section 4 with reproduction steps and evidence.

ID	Finding	Severity	OWASP 2021
F-01	SQL Injection — authentication bypass	Critical	A03 Injection
F-02	Unauthenticated admin panel access	High	A01 Broken Access Control
F-03	IDOR on shopping basket	High	A01 Broken Access Control
F-04	Sensitive file directory exposed	High	A05 Security Misconfiguration
F-05	DOM-based Cross-Site Scripting	Medium	A03 Injection

3. Scope and Methodology

3.1 Scope

Target: `OWASP Juice Shop` : storefront, REST API and administration panel, running locally at `localhost:3000`.

Environment: OWASP Juice Shop deployed via Docker inside a Kali Linux virtual machine. All browser traffic was routed through Burp Suite so every request could be inspected and replayed.

3.2 Methodology

Testing followed the OWASP Web Security Testing Guide (WSTG) and a standard penetration-testing workflow:

1. **Reconnaissance:** reviewing robots.txt, mapping pages and API endpoints.
2. **Manual testing:** authentication, access control, input validation and business logic via Burp Proxy and Repeater.
3. **Exploitation:** safe, non-destructive proof of concept to confirm impact.
4. **Reporting:** each finding mapped to the OWASP Top 10 (2021) and rated with CVSS 3.1.

3.3 Tools

Burp Suite Community Edition (Proxy, Repeater), Chromium, Docker, and browser developer tools. A step-by-step record of the test environment is provided in Appendix A.

3.4 Severity Ratings

Each finding is rated using the Common Vulnerability Scoring System (CVSS) v3.1 and classified into the qualitative bands below.

Severity	CVSS Score	Description
Critical	9.0 – 10.0	Trivially exploitable; leads to full system or data compromise. Fix immediately.
High	7.0 – 8.9	Serious impact such as data exposure or account takeover. Fix urgently.
Medium	4.0 – 6.9	Moderate impact or requires specific conditions. Fix in a planned manner.
Low	0.1 – 3.9	Limited impact; mainly assists an attacker. Fix as resources allow.

4. Technical Findings

F-01 — SQL Injection in Login (Authentication Bypass)

Severity	Critical	CVSS 3.1	9.8
OWASP	A03 Injection	Location	`POST /rest/user/login`

Description

The login form builds its SQL query by concatenating the email field directly into the statement. By injecting SQL syntax into the email field, the password check can be neutralised, allowing authentication as any user without valid credentials.

Steps to Reproduce

1. Browse to the login page.
2. In the **Email** field enter `' OR 1=1--` and any value in the password field.
3. Submit the form.

Evidence

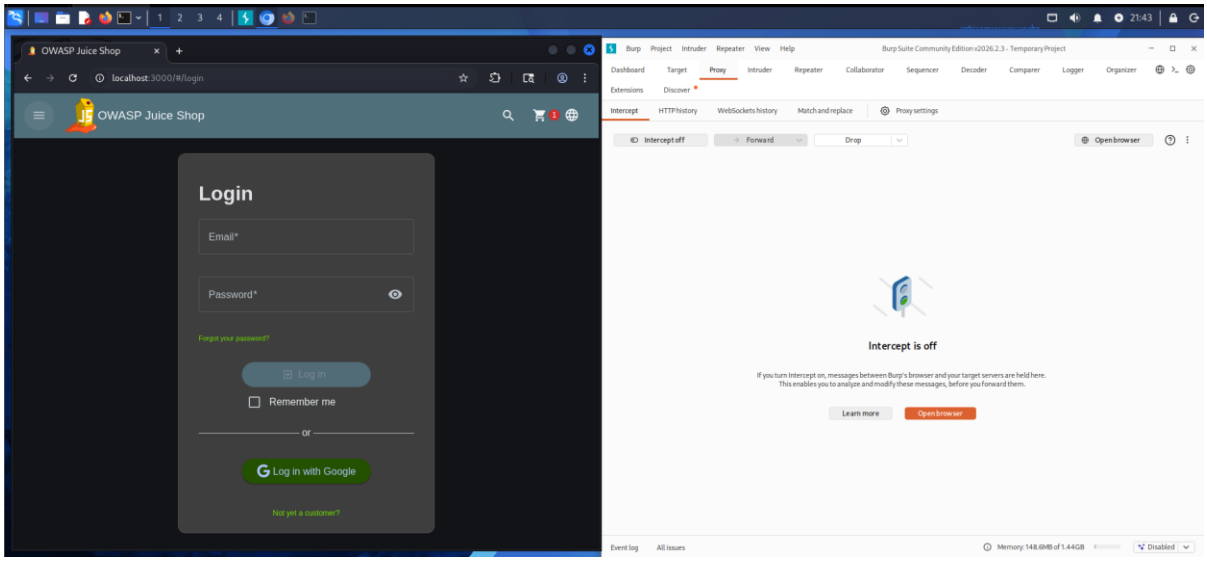


Figure 1 — Login page before the attack.

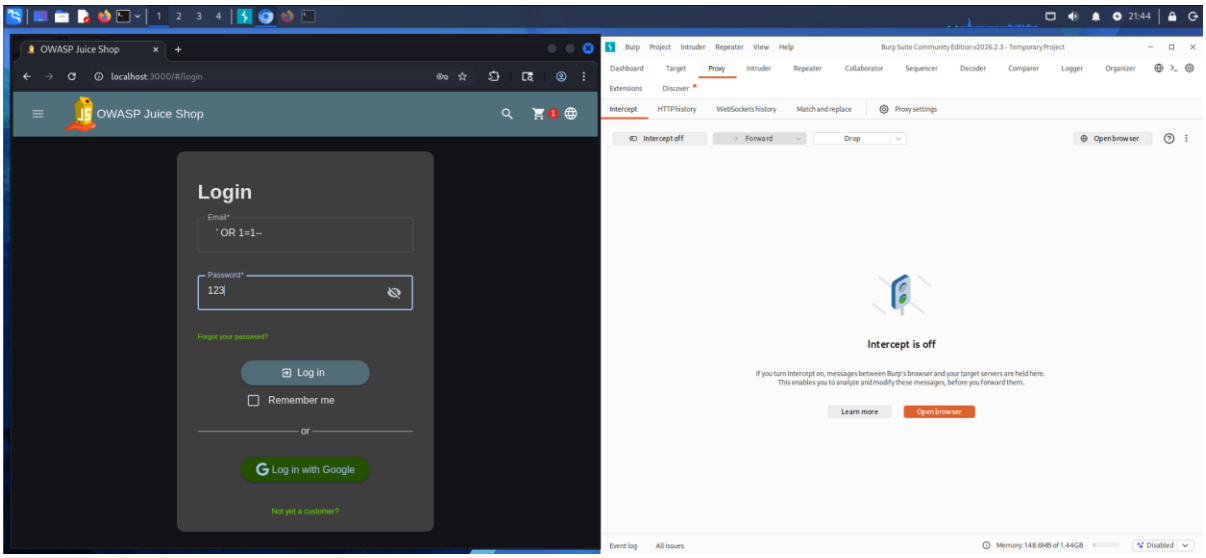


Figure 2 — Injection payload entered in the email field.

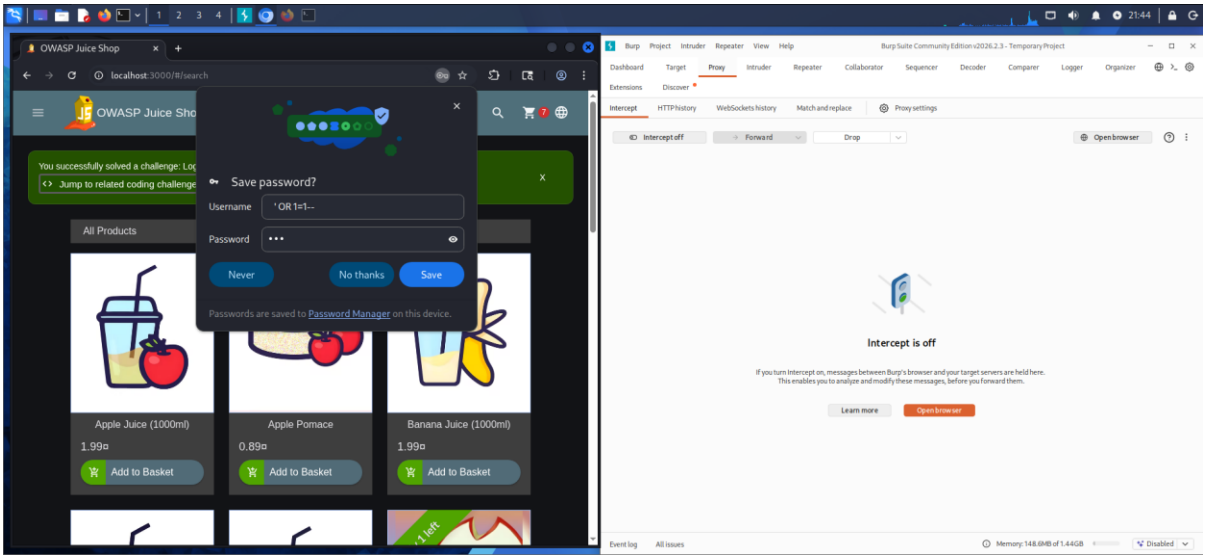


Figure 3 — Logged in as administrator; the application confirms the "Login Admin" challenge was solved.

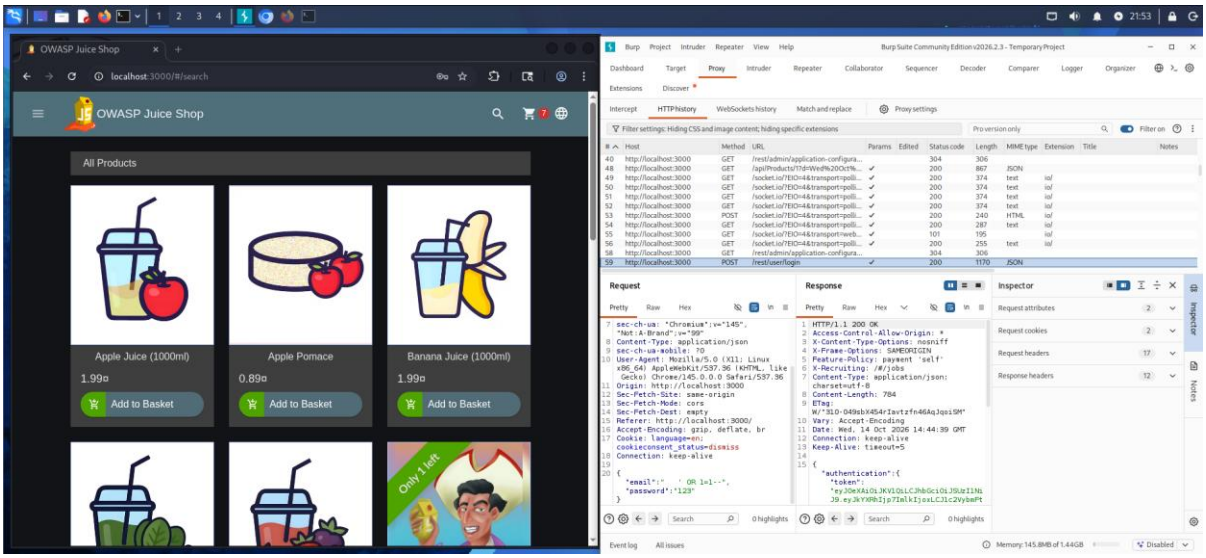


Figure 4 — Burp Suite shows the request body containing the payload and the server returning 200 OK with a valid authentication token.

Impact

An attacker can take over any account, including the administrator, without knowing any password. This leads to full compromise of customer data when chained with F-02.

Remediation

- Use parameterized queries (prepared statements) or an ORM for all database access; never concatenate user input into SQL.
- Run the application with a least-privilege database account.
- **Reference:** `CWE-89`; OWASP SQL Injection Prevention Cheat Sheet.

F-02 — Unauthenticated Access to Administration Panel

Severity	High	CVSS 3.1	8.6
OWASP	A01 Broken Access Control	Location	`/#/administration`

Description

The administration page performs no proper server-side role check. Any authenticated session — including one obtained through the SQL injection in F-01 — can reach it and view sensitive data.

Steps to Reproduce

1. Authenticate to the application (for example via the F-01 SQL injection).
2. Browse directly to `localhost:3000/#/administration`.

Evidence

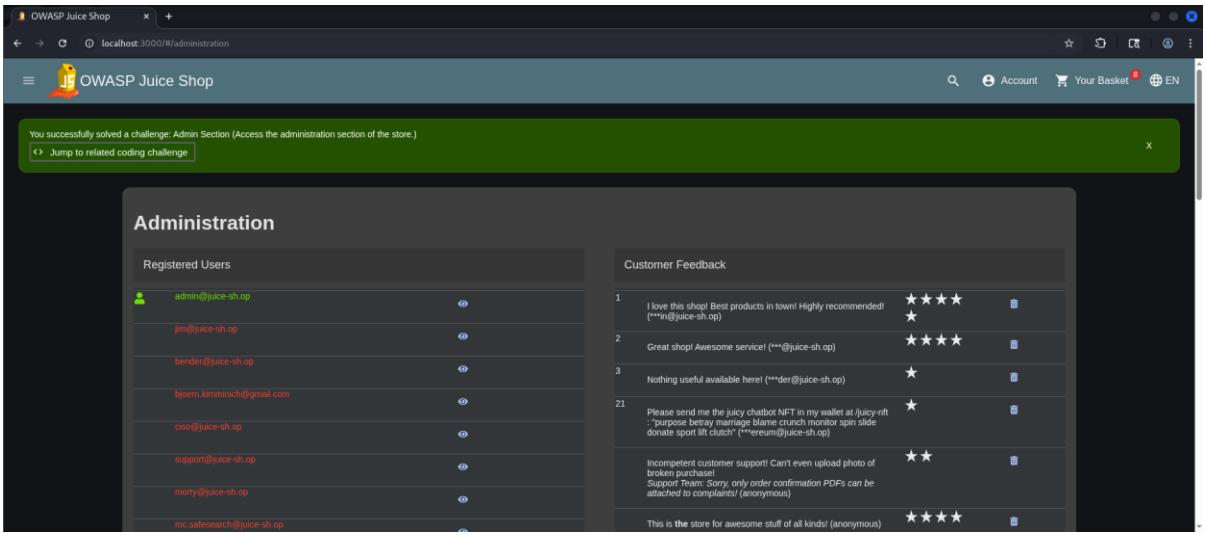


Figure 5 — The administration panel loads, exposing every registered user's email address and all customer feedback. The "Admin Section" challenge is confirmed solved.

Impact

All customer email addresses are disclosed and feedback can be managed by an unauthorized user. Combined with F-01, this represents a full breach of customer data.

Remediation

- Enforce server-side, role-based authorization on every administrative route and API endpoint — do not rely on hiding links in the UI.
- **Reference:** `CWE-284`; OWASP Broken Access Control.

F-03 — Insecure Direct Object Reference on Basket

Severity	High	CVSS 3.1	7.1
OWASP	A01 Broken Access Control	Location	`GET /rest/basket/{id}`

Description

Shopping baskets are referenced by a sequential numeric ID. The server only verifies that the user is authenticated, not that the basket belongs to them, so changing the ID returns another user's basket.

Steps to Reproduce

1. View your own basket and capture `GET /rest/basket/1` in Burp.
2. Send the request to Burp Repeater.
3. Change the ID to `2` and resend the request.

Evidence

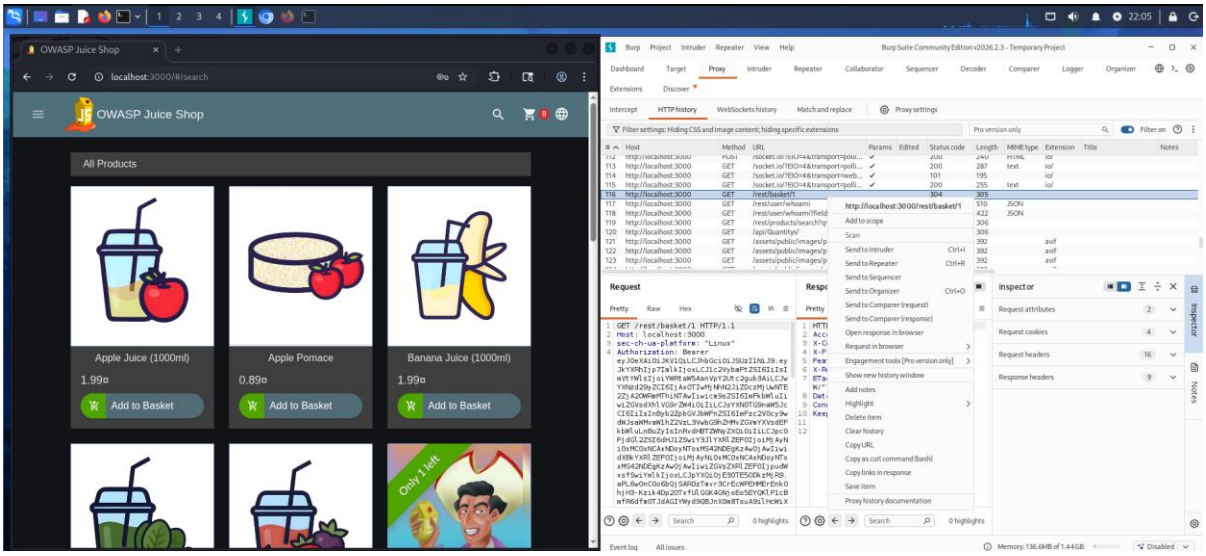


Figure 6 — Viewing the authenticated user's own basket.

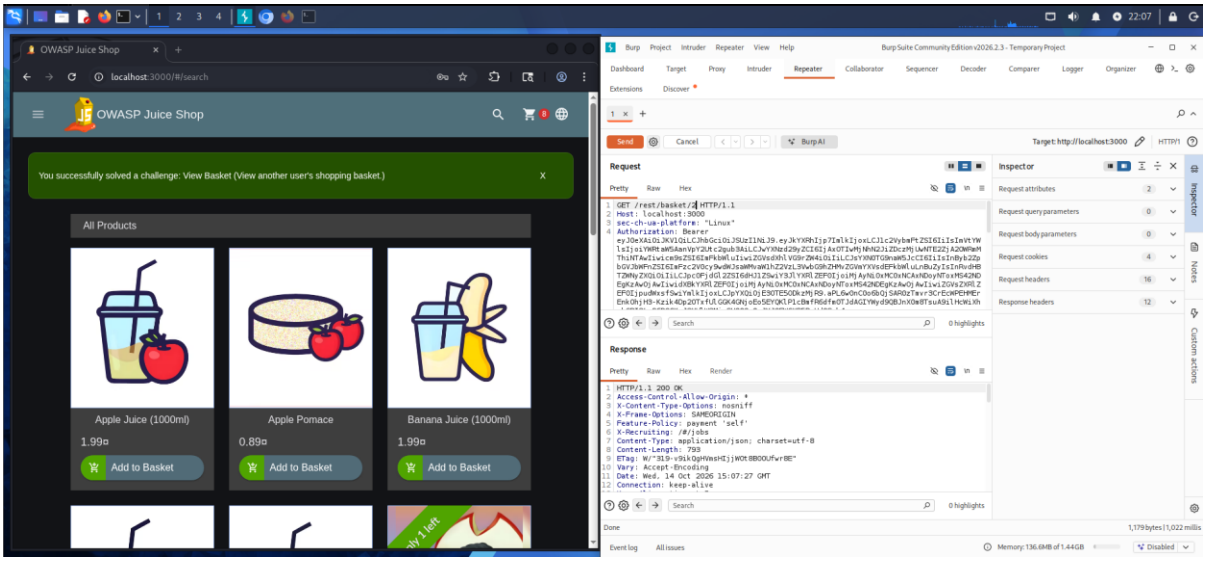


Figure 7 — In Repeater, requesting basket ID 2 returns another user's basket with HTTP 200; the "View Basket" challenge is confirmed.

Boundary testing. Out-of-range identifiers such as `-9999` and `999999` returned HTTP 200 with an empty body (Content-Length 32), confirming the endpoint processes arbitrary IDs without ownership validation.

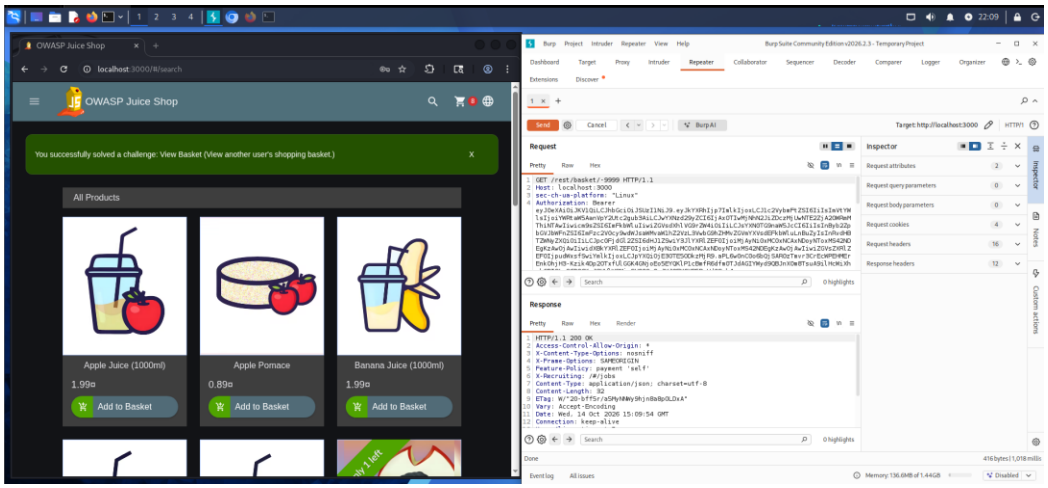


Figure 8 — Negative basket ID accepted (empty result), showing missing input and ownership checks.

Impact

Any authenticated user can enumerate and read other users' baskets and associated personal data by changing a single number.

Remediation

- Verify on the server that the requested object belongs to the authenticated user; centralise this in a shared authorization layer.
- **Reference:** `CWE-639`; OWASP IDOR Prevention Cheat Sheet.

F-04 — Sensitive File Directory Exposed via robots.txt

Severity	High	CVSS 3.1	7.5
OWASP	A05 Security Misconfiguration	Location	`/robots.txt → /ftp`

Description

The robots.txt file discloses a hidden /ftp path. That directory has directory listing enabled and exposes backup files, internal documents and a password database.

Steps to Reproduce

1. Open `localhost:3000/robots.txt` and note `Disallow: /ftp`.
2. Open `localhost:3000/ftp`.

Evidence

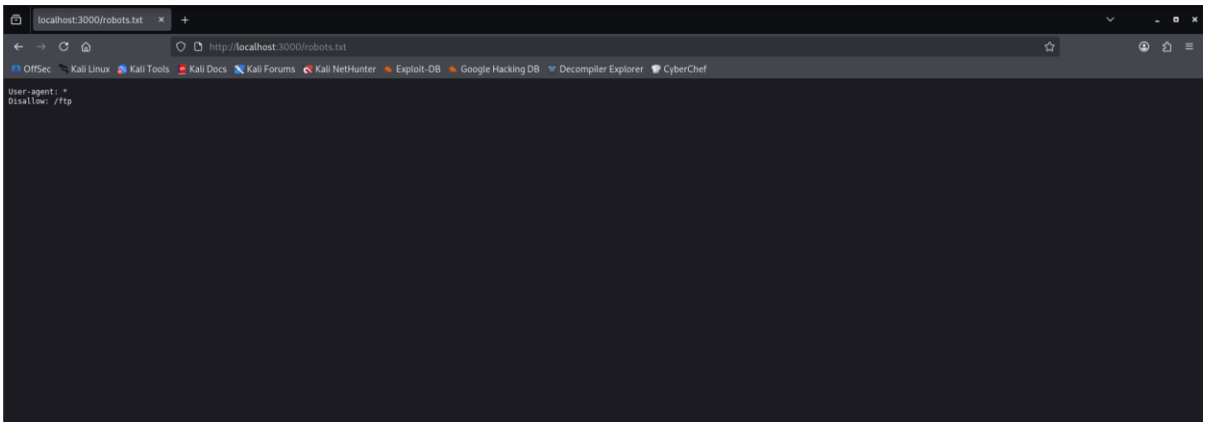


Figure 9 — robots.txt discloses the hidden /ftp directory.

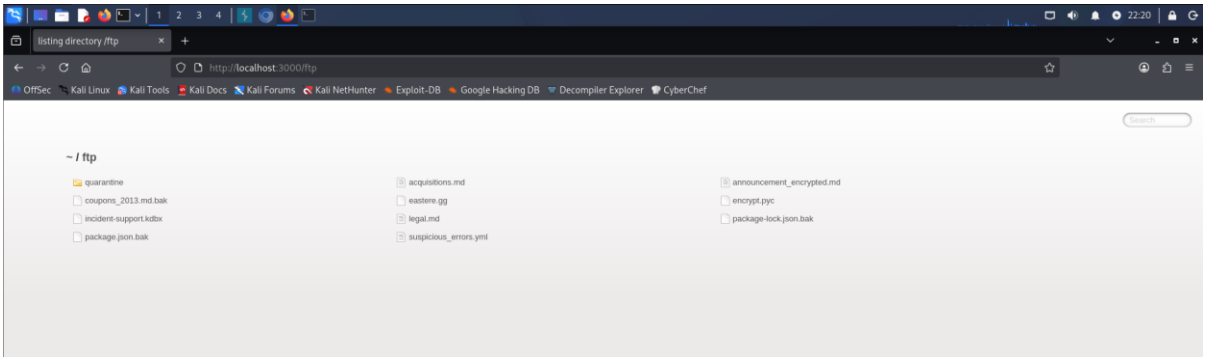


Figure 10 — The /ftp directory lists sensitive files, including .bak backups, an internal acquisitions.md, and a KeePass password database (incident-support.kdbx).

Impact

Internal documents, source backups and a password vault are retrievable without exploiting any application code, aiding further attacks.

Remediation

- Do not reference sensitive paths in robots.txt.
- Disable directory listing and move sensitive files out of the web root.
- **Reference:** `CWE-548`, `CWE-530`.

F-05 — DOM-Based Cross-Site Scripting in Search

Severity	Medium	CVSS 3.1	6.1
OWASP	A03 Injection	Location	`Product search field`

Description

The product search term is rendered back into the page without output encoding. Injected HTML and JavaScript are therefore executed by the browser.

Steps to Reproduce

1. In the search box, enter `<iframe src="javascript:alert(`xss`)">`.
2. Submit the search.

Evidence

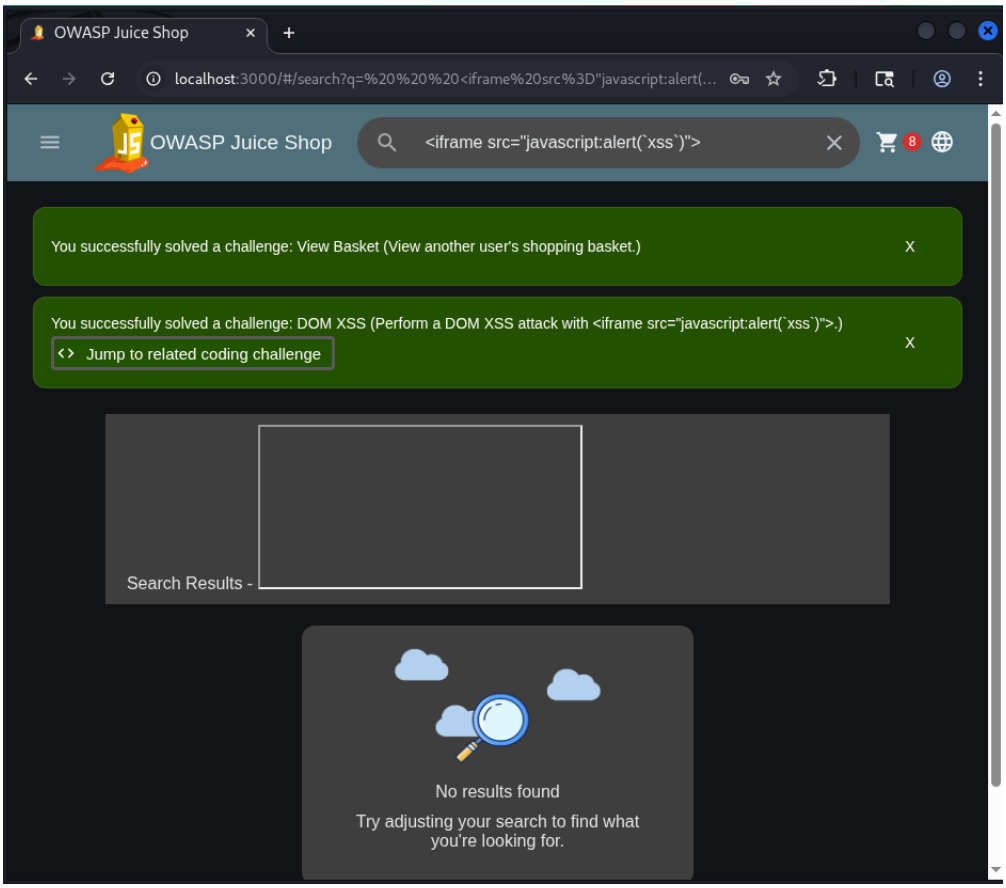


Figure 11 — The XSS payload submitted in the product search field.

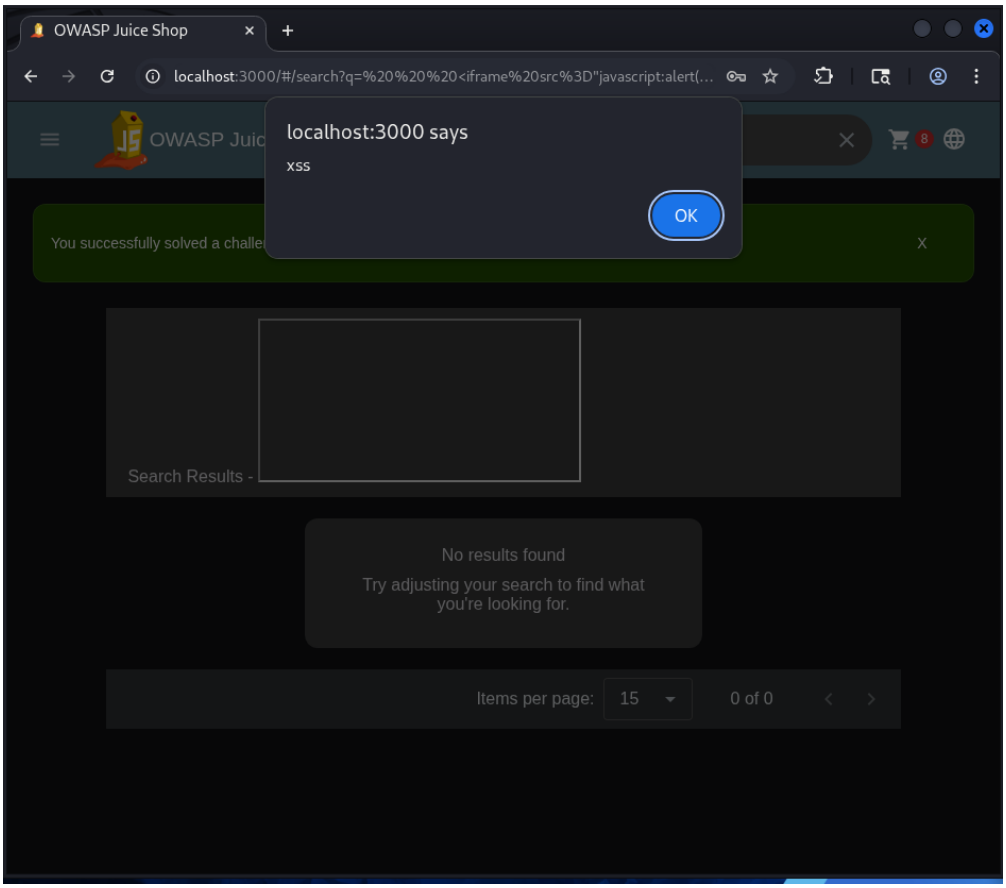


Figure 12 — A JavaScript alert executes, confirming script injection; the "DOM XSS" challenge is solved.

Impact

An attacker can execute arbitrary JavaScript in a victim's browser, enabling session theft, phishing or defacement.

Remediation

- Apply context-aware output encoding and sanitize user input.
- Deploy a strict Content Security Policy (CSP).
- **Reference:** `CWE-79`; OWASP XSS Prevention Cheat Sheet.

5. Remediation Summary

Priority	Findings	Key actions
Immediate	F-01, F-02	Parameterized queries on login; enforce server-side role checks on all admin routes.
Short term	F-03, F-04	Object-level authorization on the basket API; remove /ftp from the web root and disable directory listing.
Medium term	F-05	Output encoding and a Content Security Policy.
Ongoing	All	Add security testing to the development pipeline; retest after remediation.

Appendix A — Test Environment Setup

This appendix records how the testing lab was prepared, demonstrating a repeatable and controlled methodology.

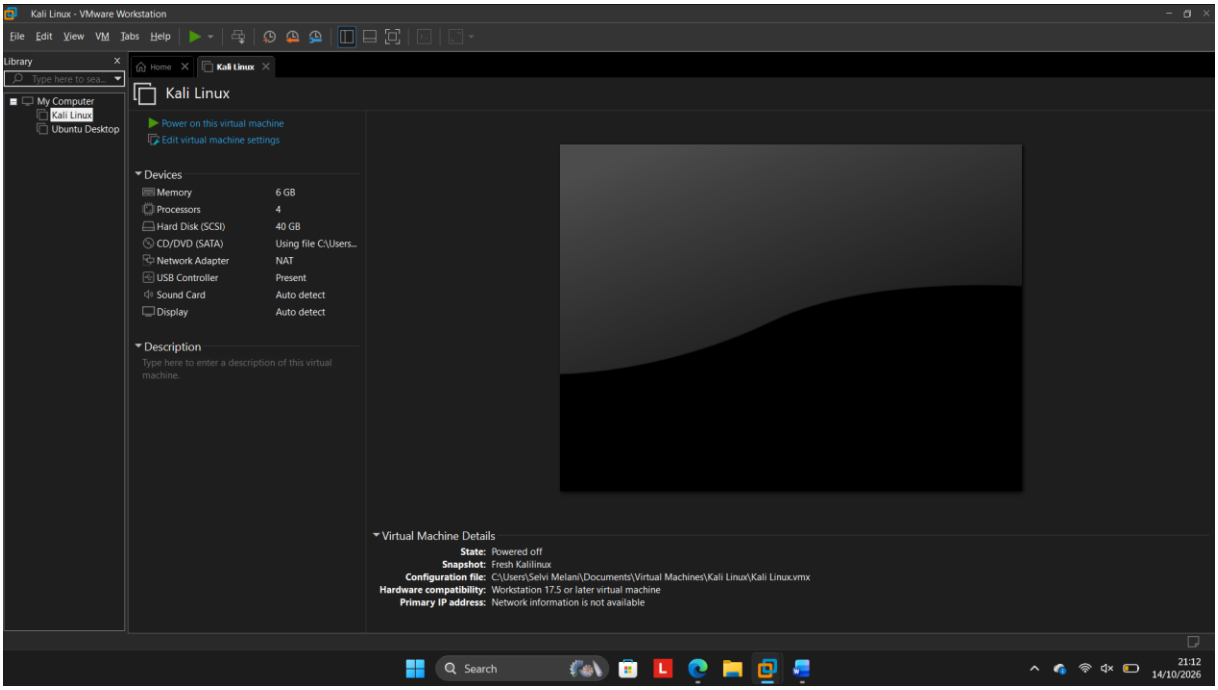


Figure A1 — Kali Linux virtual machine specification (6 GB RAM, 4 vCPU)

.

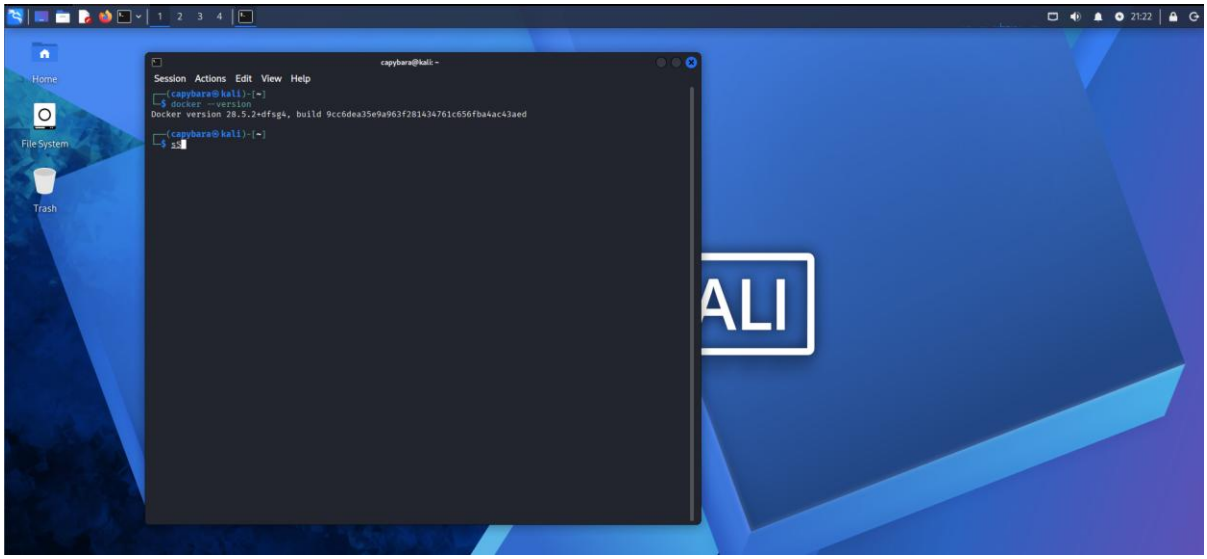


Figure A2 — Verifying Docker is installed on Kali.

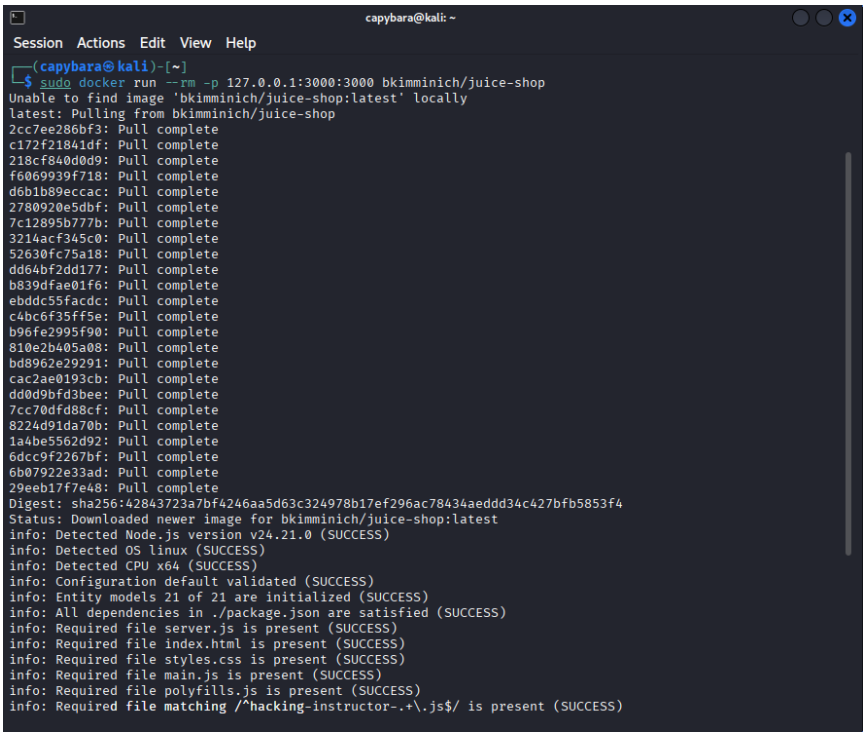


Figure A3 — Launching OWASP Juice Shop via Docker.

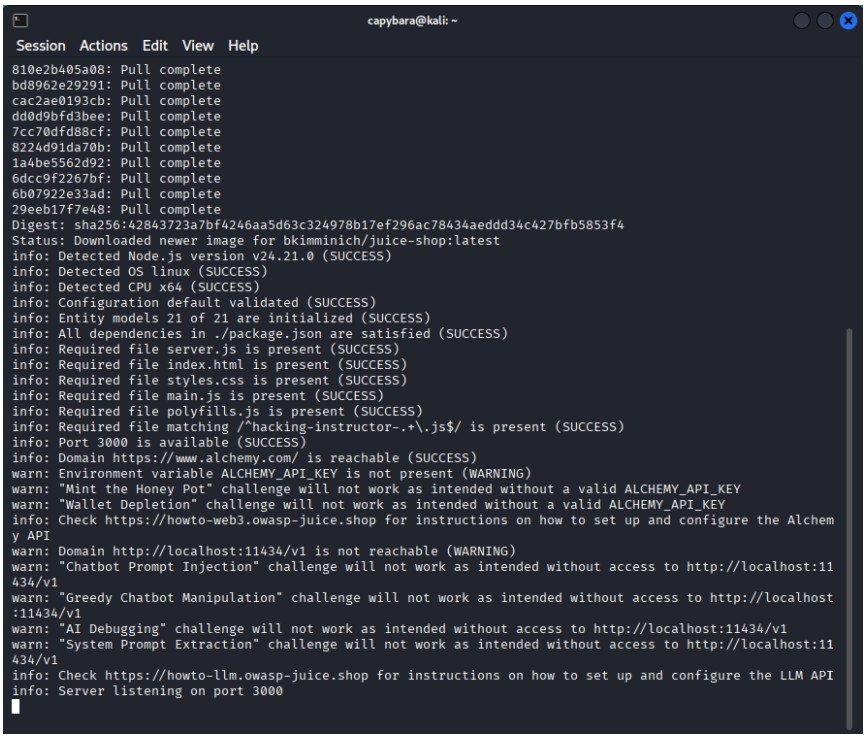


Figure A4 — Juice Shop server listening on port 3000.

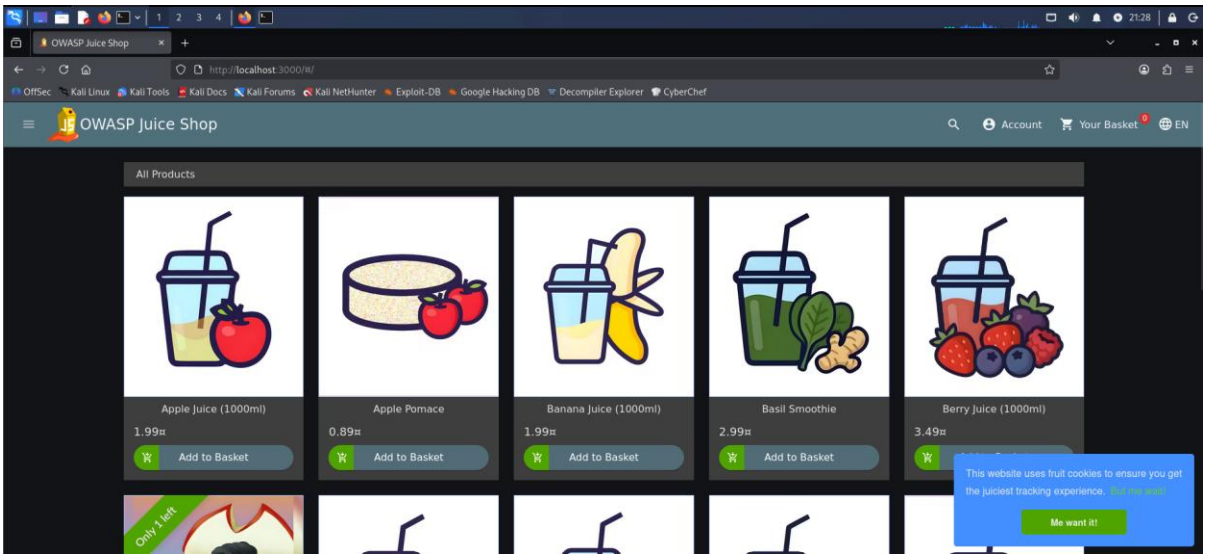


Figure A5 — Accessing the Juice Shop storefront at localhost:3000.

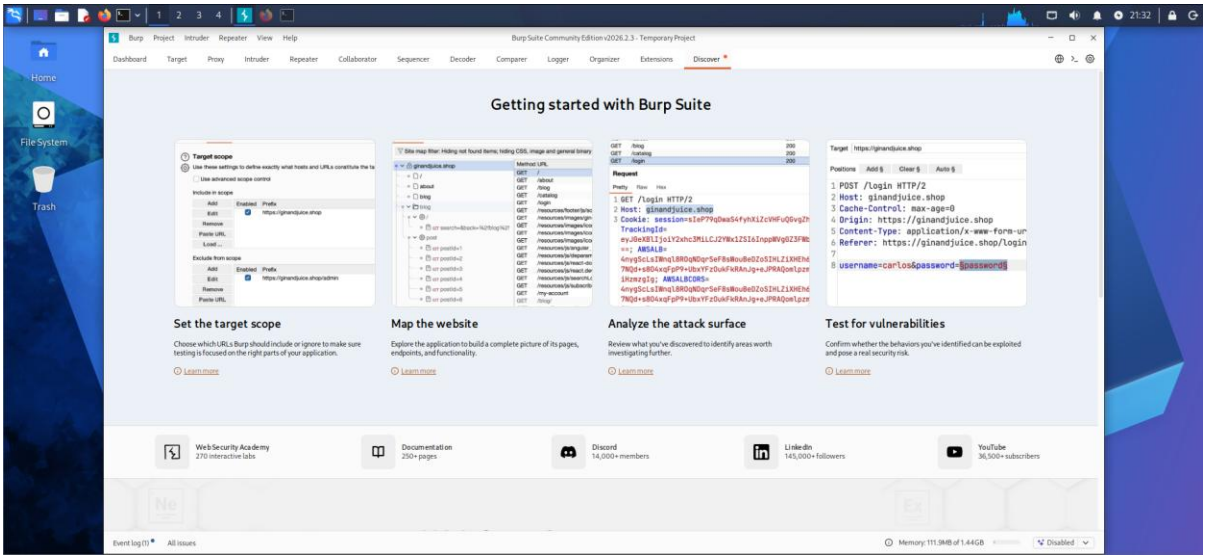


Figure A6 — Starting Burp Suite Community Edition.

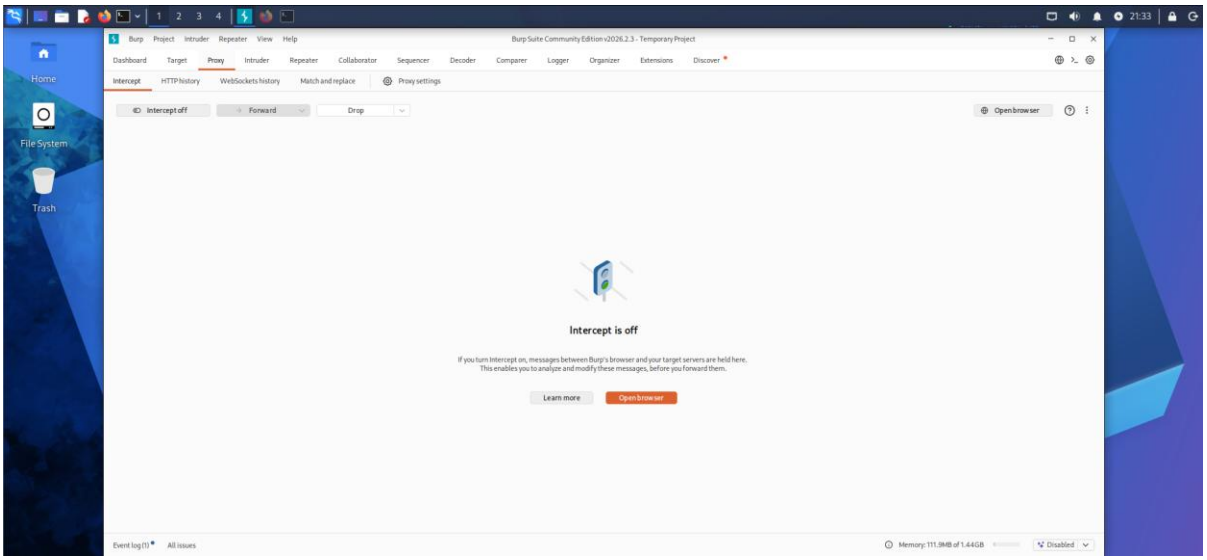


Figure A7 — Opening Burp's built-in browser.

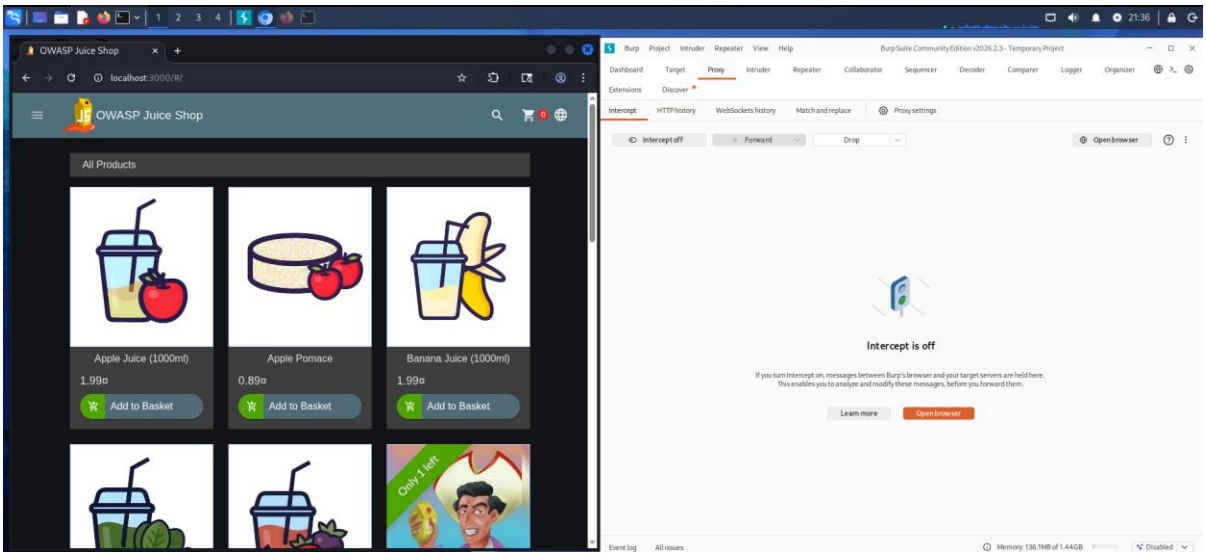


Figure A8 — Routing the Juice Shop session through Burp.

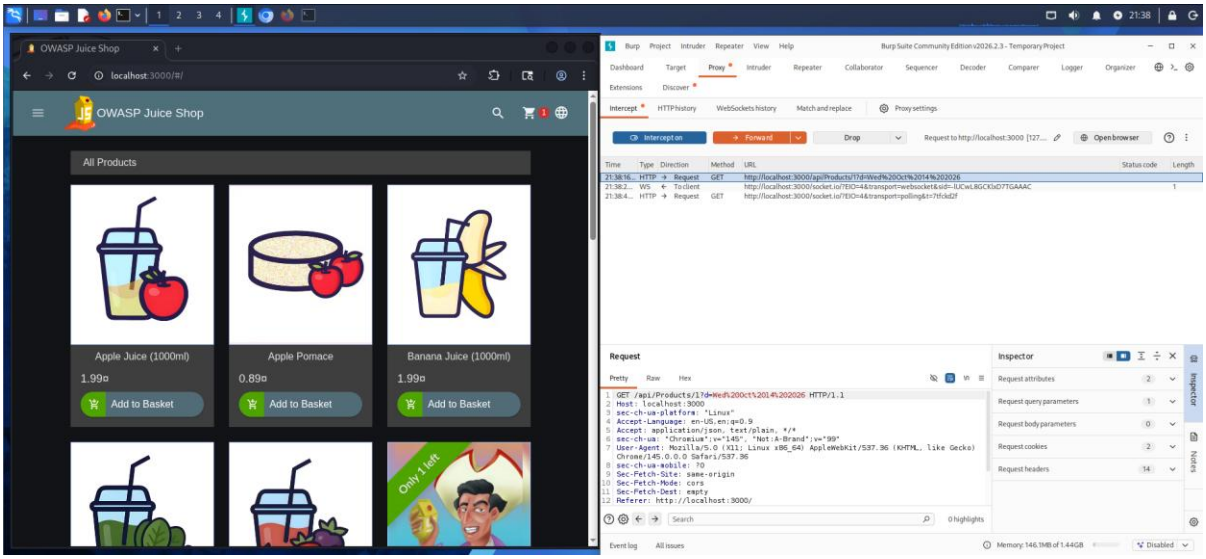


Figure A9 — First intercepted request, confirming the proxy works.

— End of Report —